

Hello!

I am Sayeed Unnisa

I am here because I love to give presentations on **Java**

You can find me at sayeedunnisa.js@gmail.com

JAVA

What is java

Java is an Object Oriented Programming Language (OOP)

*It was developed by James Gosling at Sun Microsystems in 1991.
At that time, they called it OAK.*

Oracle later acquired Sun Microsystem in 2010.

Java is the most widely used programming language.

It is designed for the distributed environment.

Features of Java

1.Simple:

- It is easy to learn, read and write in Java
- Java follows simplicity of syntax compared to C / C ++
- Java eliminates many complex and ambiguous concepts of C / C++
- Removing *Goto* statements, pointers and operator overloading

2. Object-Oriented:

- Everything in Java is in the form of the object
- A Java program must have at least one class and object
- Java strongly supports the concepts of Object-Oriented Programming
- supports features like Encapsulation, Abstraction, and Inheritance.

Features of Java

3. Robust:

- Robust means strong, It uses strong memory management
- Java provides automatic garbage collection
- Java always tries to check errors at runtime and compile time

4. Secure:

- Java is best known for its security because Java run inside a virtual machine.
- No pointers in java that avoids security problems

5. Portable:

- Run on different types of machines with no changes
- Java programs are portable, which means that the same bytecode program can run on any computer system that has a Java interpreter.

Features of Java

6. Platform-independent:

- Java provides a guarantee that to write code once and run it anywhere(at any platform)
- The compiled byte code is platform-independent, and we can run it on any machine irrespective of the Operating system

Features of Java

7. High Performance

- Java is faster than other traditional interpreted programming languages because Java byte code is "close" to native code.
- It is still a little bit slower than a compiled language (e.g., C++).
Java is an interpreted language that is why it is slower than compiled languages, e.g., C, C++, etc.

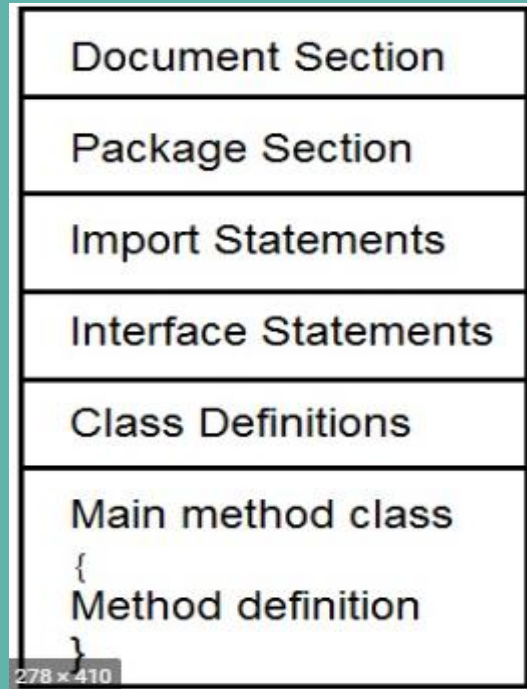
8. Distributed:

- Java is a distributed language as it provides networking facilities.
- This feature of Java makes us able to access files by calling the methods from any machine on the internet.

9. Multi-threaded:

- Java is a multi-threaded language as it can handle more than one job at a time.

Structure of Java program?



Declaration rules for a source file[.java]

- A source file can have only one public class
- A source file can have any no. of non public classes.
- If the source file contains public class then file name must match with public class name
- If the source file does not contain public class then no naming restriction to file name
- If we define a class inside the package then the package statement must be declared at first line in the source file.
- Inside a source file, import statements must be declared between package statement and class declaration.
- Note: If there is no package statement then import statement must be declared at first line.

JAVA COMMENTS :-

- ❖ Java comments are non-executable statements
- ❖ To provide the description about the program we have to use java comments.

1) Single line Comments:-

Starts with.....// (double slash)

Syntax:- //description

2) Multi line Comments:-

This comment is used to provide description about our program in more than one line.

Syntax: - /*line-1
.....line-2
*/

Data Types

- Data types are used to represent the type of the variable and type of the expression.
- Data types are used to specify the how much memory is allocated for variables.

There are 8 primitive data types

byte 1

short 2

int 4

long 8

float 4

double 8

char 2

boolean N/A

Java notations : (Java coding standards)

1. package & subpackage: all small letters. Sub packages are separated with(.) symbol

Ex: java.lang

2. Class and Interface names should starts with Upper case letter. If it contains multiple words every inner word should starts with upper case letter. No space between words

Ex: String

StringBuffer

3. variable and method: second word onwards first letter of the word is capital. No space between words

Ex. empNo

netIncome

Identifiers

A name in the program is an identifier it may be class name or method name, variable name

- 1) A java identifier is a sequence of characters, where each character may be a letter from *a-z* or *A-Z* or a digit from *0-9* or currency symbol *\$* or connecting punctuation *,* if we are using any other symbol we will get Compile time error "*IllegalCharacter*".
- 2) Identifier should not be starts with digit.
- 3) There is no length limit for java identifiers but it is not recommended to take more than 15 length.
- 4) Java Identifiers are case sensitive
5. reserve words cannot be used as identifiers.

Classes in java

What is class?

A class is a collection of variables and methods.

“A class is a way of binding the data and associated methods in a single unit”.

Any JAVA program if we want to develop then that should be developed with respect class only i.e., without class there is no JAVA program.

Syntax:

```
class class_name
{
    Datatype variable1;
    Datatype variable2;
    -----
    returntype method_name(arguments list)
    {
        ---method definition
    }
}
```

Classes in java

What is Object?

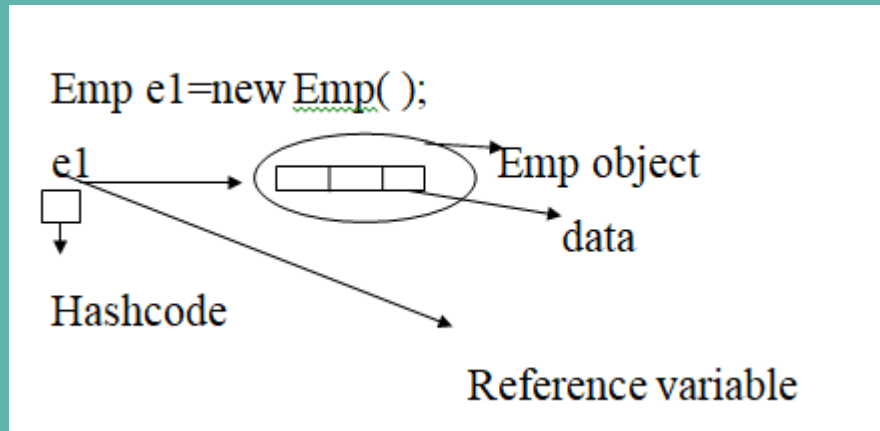
An instance of a class is called object.

Classname objectname = new classname();

Ex: Demo ob=new Demo();

“ob” is called as Object name or Reference variable or Object Reference

“new” is called dynamic memory allocation operator. Because it allocates the memory at runtime.



variable

A variable is a container which contains data

A Java variable is a piece of memory that can contain a data value.

every variable has a data type .

Declaration:

`datatype variable;`

Ex: `int a;`

Assignment:

`Variable=literal;`

Ex `a=10;`

Variable:

Initialization:

Datatype variable=literal;

Ex:

```
int x=10;
```

Note: primitive type variable stores data where as reference variable stores hashcode

Types of variables:

- ▣ **Instance variables**
- ▣ **Static variables**
- ▣ **Local variables**

Instance variables:

- If the value of the variable is varied from object to object such type of variables are called instance variable.
- The scope of the instance variable is exactly same as the object because instance variables are created at the time of object creation and will be destroyed at the time of object destruction.
- Instance variables should be declared inside the class and outside the method or constructors or a block.
- Instance variables cannot be accessed from static area directly we can access it by using object or object reference.
- A variable which is defined as a member of a class is known as an instance variable
- Memory allocated to instance variable whenever an object is created
- Instance variables are stored in heap area.
- Separate copy of instance variable exists for every instance

Instance variables:

There are two ways to access instance variables

1.by using object

2.by using object reference

Use object to access instance variable if it is required only one time

Use object reference to access instance variable if it is required more than one time

For instance variables it is not required to perform initialization explicitly , jvm will provide default values

Static variables:

- A variable which is defined inside a class and outside the method or a constructor or block with static modifier is called as static variable or class variable.
- Memory allocated to class variables when class is loaded .
- Class variables are stored in method area.
- The value of static variable is not varies from object to object, for all the objects only one copy of static variable is available.
- Static variables are called as class level variables.
- Static variables are created at the time of class loading and destroyed at the time of class is unloading.
- Hence the scope of the static variable is exactly same as the scope of the class.

Static variables:

- Static variables are created at the time of class loading i.e.(at the beginning of the program) hence we can access from both instance and static area directly
- For the static variables it is not required to perform initialization explicitly ,jvm will provide default values.

There are 4 ways to access class variables

1.directly

2.by using class name

3.by using object

4.by using object reference

Note: Last two ways are not recommended

Static variables:

- Access class variables directly if they are present in the same class.
- Use class name to access the static variables if they are present in the different class.
- If we perform any changes to the static variables using one object it will reflect in remaining objects also because single copy will be shared by all the objects

Local variables:

- The variables which are defined inside the method or a block or constructors , such type of variables are called as local variables.
- Memory is allocated to local variables when ever a method/block/constructor is called .
- Local are stored in stack area of the memory.
- Note: local variables cannot be static in java.
- There is only one way to access the local variables i.e. directly.
- For the local variable jvm wont provide any default values, compulsory we should perform initialization explicitly, before using that variable.
- The only applicable modifier for local variables is final.

Operators:

- ❖ It is a symbol which perform a particular operation.
- ❖ An operator is a symbol that operates on one or more Operands to produce a result.
- ❖ An operands are the values on which the operators act upon.
- ❖ Java has a rich set of operators. The following operators available in Java are

Operators:

- 1) Increment and decrement operator
- 2) Arithmetic operators
- 3) Relational operators
- 4) Logical Operators
- 5) Ternary Operator (? :)

Increment and decrement operator

The increment operator is used to increment (adds) the value of operand/variable by 1, whereas the decrement operator is used to decrement (subtracts) the value of operand or variable by 1

Increment/Decrement Operators		Let us assume X is a variable
Operator	Expression	Description
++	++X	Pre-increment
	X++	Post-increment
--	--X	Pre-decrement
	X--	Post-decrement

Arithmetic operators

Arithmetic operators in java are used to perform calculations like addition, subtraction, multiplication, on variables and data.

Operator	Meaning	Description
+	Addition	Adds two operands
-	Subtraction	Subtracts right operand from left operand
*	Multiplication	Multiplies two operands
/	Division	Divides left operand by right operand
%	Modulus	Remainder after division

Relational operators

- ✓ Relational operators do compare the data provided on the LHS and RHS to give binary outputs i.e. True or False.
- ✓ These are used to test the relation between two values or two variables or expressions.

Operator	Meaning	Description
==	Equal to	Returns True if two operands are equal
!=	Not Equal to	Returns True if two operands are not equal
>	Greater than	Returns True if left operands is greater than the right
<	Less than	Returns True if left operand is less than the right
>=	Greater than or equal to	Returns True if left operands is greater than or equal to the right
<=	Less than or equal to	Returns True if left operand is less than or equal to the right

Logical Operators

- ✓ These operators are used for combine the result of two or more expressions.
- ✓ These logical operators work only on boolean operands. Their return values are always boolean.

Logical Operator	Java Operator
AND	&&
OR	
NOT	!

Control Statements

A control statement in java is a statement that determines whether the other statements will be executed or not.

There are 4 control statements

Simple if

If else

Nested if

switch

Looping statements:

loops are used to repeat a block of code.

A program loop is **a series of statements that executes for a specified number of repetitions or until specified conditions are met.**

In Java, there are three types of loops.

- ▣ for loop
- ▣ while loop
- ▣ do...while loop

for Loop

Java for loop is used to run a block of code for a certain number of times. The syntax of for loop is:

```
for (initialExpression; testExpression; updateExpression) {  
    // body of the loop  
}
```

while loop

Java while loop is used to run a specific code until a certain condition is met. The syntax of the while loop is:

```
while (testExpression) {  
    // body of loop  
}
```

do-while loop

Java do-while loop is called an exit control loop.

the do-while check the condition at the end of loop body.

The Java do-while loop is executed at least once because condition is checked after loop body.

```
do{
```

```
// loop body
```

```
}while (condition);
```

Methods:

A **method** is a block of code which only runs when it is called. You can pass data, known as parameters, into a **method**. **Methods** are used to perform certain actions

Categories of methods:

1. Methods with arguments and with return value
2. Methods with arguments and without return value
3. Methods without arguments and with return value
4. Methods without arguments and without return value

Polymorphism:

The ability to take more than one form is called polymorphism

Poly means many, morph means form

There are two types of polymorphism

1.compile time polymorphism(static binding)

2.run-time polymorphism(dynamic binding)

1.compile time polymorphism

Binding of method call statement with method definition is done at compile time only is known as compile time polymorphism

Ex: method overloading

Method Over loading:

If two or more methods with the same name and with different parameters list then it is said to be method overloading

In a parameter list difference can be a number of arguments or datatypes or order of arguments

2.run-time polymorphism

Binding of method call statement with method definition is done at runtime only is known as run-time polymorphism

Ex: method overriding

Method Overriding:

If two or more methods with the same name and with the same parameters list then it is said to be method overriding.

Methods can not be overridden in the same class because of ambiguity.

Parameter passing in methods

- ❑ Information can be passed to methods as parameter.
- ❑ Parameters act as variables inside the method.
- ❑ There are basically two types of techniques for passing the parameters in Java.
 1. **pass-by-value**
 2. **pass-by-reference.**
- ❑ One thing to remember here is that when a primitive type is passed to a method, then it is done by the use of pass-by-value.
- ❑ all the non-primitive types that include objects of any class are always implicitly passed by use of pass-by-reference.

Constructors:

- Constructor is a special method which has the same name as the class name .
- Constructor does not have any return type
- Constructor is called automatically when ever an instance is created for a class .
- Constructors are used to initialize the instance variables of a class.

Constructors are of two types

1.default constructors

2.parameterised constructors

'this' Key word:

- It is called as reference variable because it refers an object .
- it always refers current object
- 'this' keyword is explicitly required to access instance variables when ever both instance variable and local variable names are same.
- **this()**-This statement calls default constructor of current class
- **this(----)** -This statement calls parameterized constructor of current class
- Note: **this()** and **this(----)** must be first statement in a constructor

Static methods:

- This is also called as class method
- A method which is defined with “static” keyword is known as “class method”

“Static” keyword:

- It is called as modifier because it modifies the behaviour of a variable, block, method and class.
- It is used to create class variables, static initialization class(it is one type of inner class)

Static block:

- a block with static keyword is called as static block
- one class can have any number of blocks and all those blocks are executed from top to bottom order when ever class is loaded
- static blocks are executed before main() method
- a static block is also called as static initialization block because static blocks can be used to initialize class variables

block:

- A group of statements between open brace { and } close brace is called block
- one class can have any number of blocks and all those blocks are executed from top to bottom order when ever an object is created
- Note: blocks are executed before constructor itself.
- A block is also called as initialization block because block can be used to initialize instance variables

Encapsulation:

- Binding of variables and methods into a class known as an encapsulation
- It facilitates binding of variables with methods and those methods operating on same variables

Inheritance:

- ✓ It is also called as “IS-A Relation ship”
- ✓ creating a new class from an existing class is called as inheritance
- ✓ In inheritance, existing class is said to be super class and new class is said to be sub class

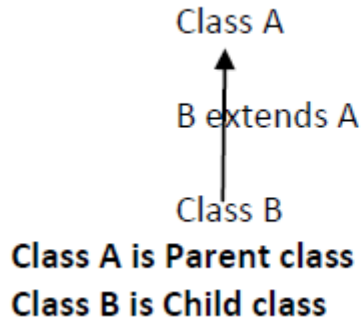
Types of Inheritance:

- ✓ Single Inheritance
- ✓ Multiple Inheritance
- ✓ Multilevel Inheritance
- ✓ Hierarchical Inheritance
- ✓ Hybrid Inheritance

Single Inheritance

- ✓ In single inheritance, a single subclass extends from a single superclass.

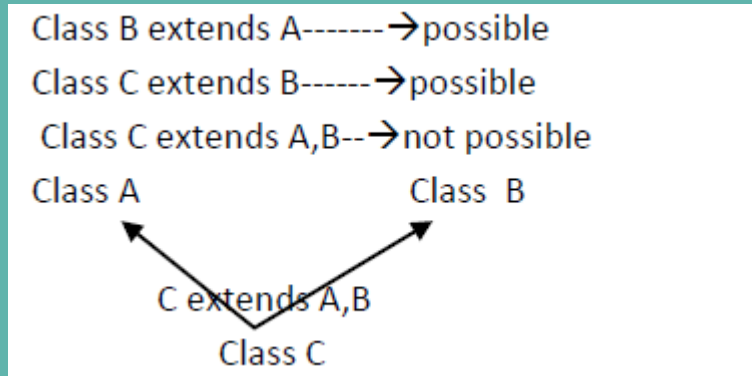
Single inheritance:-



Extending the properties from single parent class to single child class is called single inheritance.

Multiple Inheritance

- ✓ In multiple inheritance, a single subclass extends from multiple super classes.
- ✓ Note: Java doesn't support multiple inheritance. However, we can achieve multiple inheritance using interfaces.



Multilevel Inheritance

- ✓ In multilevel inheritance, a subclass extends from a superclass and then the same subclass acts as a superclass for another class.

Multilevel inheritance:-

Class A



B extends A

Class B



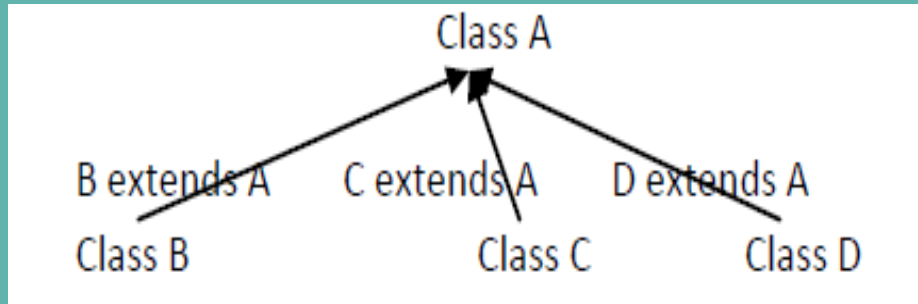
C extends B

Class C

one super and one sub class at each and every level.

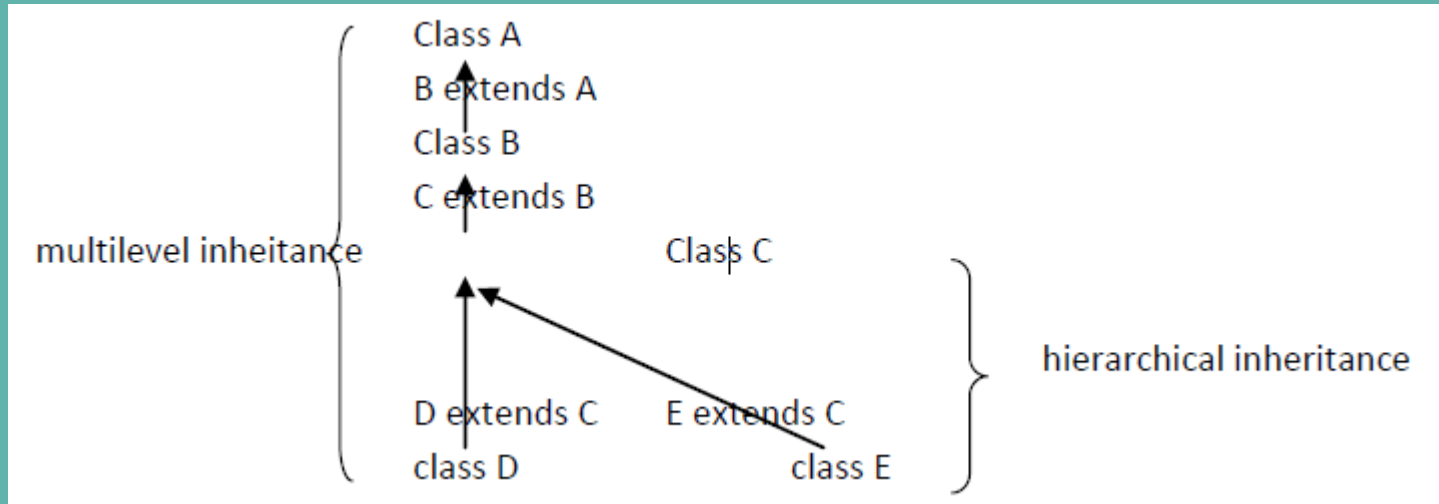
Hierarchical Inheritance

- ✓ In hierarchical inheritance, multiple subclasses extend from a single superclass.



Hybrid Inheritance

- ✓ Hybrid inheritance is a combination of two or more types of inheritance.



HAS-A Relationship:

- ✓ HAS-A relationship is also known as composition (or) aggregation.
- ✓ Creating an instance of one class as data member of another class is known as HAS-A relation ship

```
class A
```

```
{
```

```
}
```

```
class B
```

```
{
```

```
    A a; //---→ HAS-A relation ship
```

```
}
```

‘final’ key word:

- ✓ It is called as modifier because it modifies the behavior of a variable, method and class
- ✓ For final variables we cannot re-assign a value. Final variables acts like constants in java
- ✓ If a method is not allowed to override in the child class then we should declare that method as ‘final’.
- ✓ For final methods overriding is not possible.
- ✓ For any class if we are not allowing to create child class, such type of classes we have to declare with final
- ✓ For final classes inheritance is not possible.

Arrays:

- An array is a collection of similar data elements
- Java array is an object that contains elements of similar data type.
- The main advantage of arrays is we can represent a group of values with single variable name
- The main limitation of arrays is they are fixed in size. i.e once we constructed an array there is no chance of increasing or decreasing based on our requirement

Arrays:

Array creation:

```
datatype Arrayname[ ] = new datatype [size];  
int a [ ] = new int[10];
```

Array initialization:

```
int [] a = {10,20,30};
```

Array length:

length is a final variable which is used to find the length(no of characters of an array)

For each loop:

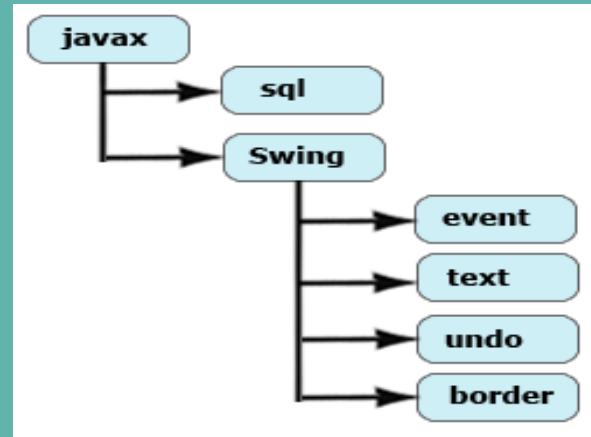
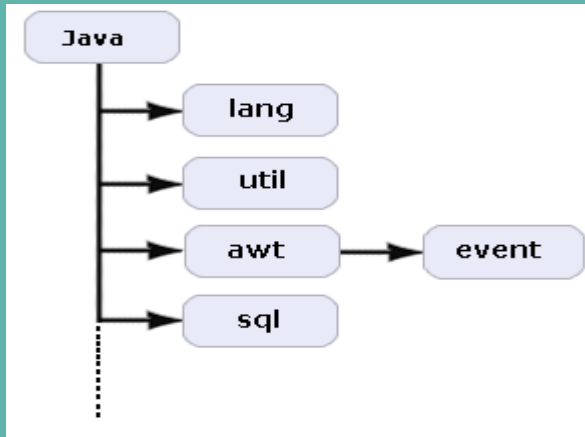
- For each Introduced in 1.5 version.
- It is also called as “enhanced for loop”
- **for-each loop** is used to iterate the elements of arrays and collections

Syntax:

```
for( dataType   tempvar : array/collection)
{
    ...
}
```

Packages:

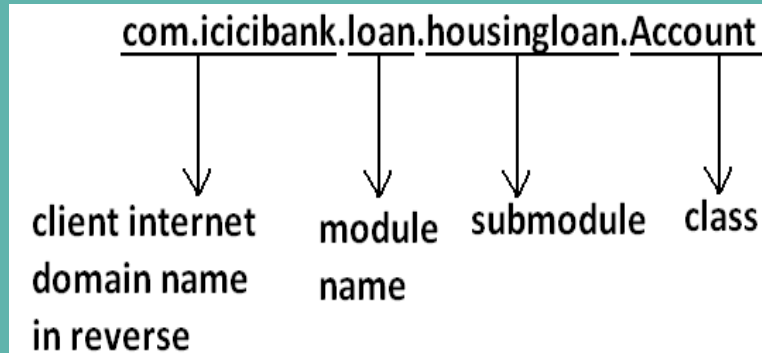
- ✓ A package is collection of sub package ,classes, Interface.
- ✓ java library organized in the form of packages.
- ✓ It is an encapsulation mechanism to group related classes and interfaces into a single module.



Packages:

The main purpose of packages are

1. To resolve naming conflicts of classes and interfaces
 2. To provide security to the classes and interfaces so that outside person can not access directly
 3. It improves modularity of the application
- there is one universally accepted convention to name packages i.e. to use internet domain names in reverse



Access Modifiers:

- ✓ They specify access permissions to variables, methods, classes and interfaces
- ✓ there are four access modifiers in java
 - 1.private
 - 2.protected
 - 3.public
 - 4.default (no name)

Public:

- This is the modifier applicable for classes, methods and variables (only for instance and static variables but not for local variables).
- If a class is declared with public modifier then we can access that class from anywhere (within the package and outside of the package)

Default:

- This is the modifier applicable for classes, methods and variables (only for instance and static variables but not for local variables).
- If a class is declared with <default> modifier then we can access that class only within that current package but not from outside of the package.
- Default access also known as a package level access.
- The default modifier in java language is **default (No Name)**.

Private:

- private is a modifier applicable for methods and variables.
- If a member declared as private then we can access that member only from within the current class.

Protected :

- If a member declared as protected then we can access that member with in the current package anywhere but outside package only in child classes.

Java.lang package

- For writing any java program the most commonly required classes and interfaces are provided in the separate package which is nothing but java.lang package.
- It is not required to import java.lang package in our program because it is available by default to every java program.
- This java.lang package contains Object, Thread, String etc.
- **Object class is the ultimate super class of all classes in Java.**

The following are some of important classes present in java.lang package.

1. Object class
2. String class
3. StringBuffer class
4. Wrapper classes

Java.lang.Object class:

- If our class doesn't extend any other class then it is the direct child class of object
- If our class extends any other class then it is the indirect child class of Object.
- the most commonly required methods are encapsulated into Object class

Strings:

- In Java, a string is a sequence of characters.
- String is a final class it is present in java.lang package.
- Once we are creating String object it is not possible to do the modifications on existing object it is called immutability nature.
- In String class .equals() is used for content comparison.

There are four string handling classes are available in java to handle strings

1. java.lang.String
2. java.lang.StringBuffer
3. java.lang.StringBuilder

There are two ways to create the strings in java

1.String s1="hello";

2.String s2=new String("welcome");

String class Constructors:

1.String ();

2.String (String);

Constructor-1 is used for creating empty string object.

Constructor-2 is used for creating a String object by taking another string as a parameter.

3.public String(char[]);

4.public String(char[], int, int); //first int indicates the starting index and second int represent no of characters from the starting index

Methods:

1. `public int length();`

Returns the length of this string.(no of characters present in the given string)

2. `public char charAt(int);`

Returns the char value at the specified index.

3. `public byte[] getBytes();`

it is used to convert the given String into bytes array

4. `public boolean equals(Object);`

Compares this string to the specified object(string)

5. `public boolean equalsIgnoreCase(String);`

Compares this String to another String, ignoring case considerations.

6. `public String concat(String);`

Concatenates (adds) the specified string to the end of this string.

7. `public String toLowerCase();`

Converts the given string to the lower case

8. `public String toUpperCase();`

converts the given string to the upper case

9. `public String trim();`

It is used to remove the extra spaces at the beginning of the string and at the end of the string

10. `public char[] toCharArray();`

It is used to convert the given string into the character array

11. `public String substring (int start, int end);`

It is used for obtaining those characters by specifying starting position to ending position.

- whenever an object or object reference is passed in `System.out.print()` method then implicitly `toString()` method is called.
- An `Object` class `toString()` method always returns `ClassName@HashCode-IN-HexaDecimalFormat`.
- where as the `String` class `toString()` method always returns the content of `String` object

StringBuffer:

- 1) String Buffer is a class present in the java.lang package.
- 2) StringBuffer is a final class so it can't be inherited.
- 3) StringBuffer is a mutable class so it is possible to make the changes in the contents of the StringBuffer object.
- 4) StringBuffer .equals() method is used for reference comparison.

Constructors:

1. StringBuffer ()
2. StringBuffer (String)

For example:

```
StringBuffer sb=new StringBuffer ();  
System.out.println (sb.capacity ());
```

Constructor-2 is used for converting String object into StringBuffer object.

For example:

```
String s="HELLO";
```

```
StringBuffer sb=new StringBuffer (s);
```

methods:

1. public int length ();

This method is used for determining the length of the string.

2. public int capacity ();

This method is used for determining the capacity of StringBuffer object. Capacity of StringBuffer object is equal to the number of characters in the StringBuffer plus 16 additional characters memory space.

3. public StringBuffer reverse ();

This method is used for obtaining reverse of source StringBuffer object.

4. `public void append (String);`

This method is used to append string data at the end of source StringBuffer object.

5. `public StringBuffer delete (int start, int end);`

This method is used for removing the specified number of characters from one position to another position.(in between characters will be deleted from starting index to the ending index)

6. `public StringBuffer replace (String, int, int);`

This method is used for replacing the string into StringBuffer object from one specified position to another specified position.

7. `public StringBuffer insert (String, int, int);`

This method is used for inserting the string data from one specified position to another specified position.

Abstract class:

- A class that is declare with abstract keyword is know as abstract class
- An abstract class can have only abstract methods or only non abstract methods(concrete methods) or both abstract and non abstract methods
- Abstract classes can not be instantiated
- Abstract class can be inherited into another class by using extends keyword
- when ever abstract class inherited then all abstract methods of an abstract class must be overridden in sub class otherwise sub class must be declared with abstract keyword.

Abstract method:

- A method which has no body is called as an abstract method
- Abstract method must be declared with abstract keyword in java otherwise compile time error occurs
- If the class contains an abstract method then that class must be declared with an abstract keyword otherwise compile time error occurs.
- Note: Abstract method cannot be “final”, “static” and cannot be private

Interfaces:

- An interface is basically kind of class.
- An interface is a collection of public static final variables and public abstract Methods
- In interface all variables are implicitly public static final and all methods are implicitly public abstract
- Interface implicitly itself abstract
- Interface can not be private and cannot be protected
- Interfaces can not be instantiated
- Interface can be inherited into a class by using "implements" keyword
- Interface can be inherited into another interface by using "extends" keyword

Interfaces:

- whenever an interface is inherited in to a class then all methods of an interface must be overridden or sub class must be declare as abstract class other wise compile-time error occurs
- Interface can not have constructors(implicitly and Explicitly)

Wrapper classes:

- All the primitive data types has its corresponding classes, and those classes are called wrapper classes because they wrap the data into an object
- The wrapper classes in Java are used to convert primitive types (int, char, float, etc) into corresponding objects.
- To Wrap primitives into object form. So that we can handle primitives also just like objects.
- All wrapper classes belongs to java.lang package

Primitive data types	Wrapper class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

Autoboxing and Auto Unboxing:

Autoboxing:

Automatic conversion of primitive to wrapper object by compiler is called Autoboxing.

Example:

- Integer i=10; [compiler converts “int” to “Integer” automatically by Autoboxing]

AutoUnboxing:

Automatic conversion of wrapper object to primitive by compiler is called AutoUnboxing.

Example:

Integer i=new Integer(10);

Int i=i; [compiler converts “Integer” to “int” automatically by AutoUnboxing]

Exception Handling:

Error:

A programming mistake is said to be an error

Types of errors:

1. Compile time errors(syntax errors)
2. Run time errors(exceptions)
3. Logical errors

Exception means runtime error

Exception is an unwanted and unexpected event that disturbs normal flow of the program is called exception.

If we are not handling exception, the program may terminate abnormally without releasing allocated resources(cpu,memory,I/O devices)

- All the exceptions are classes in java
- In exception handling we use the following keywords
 - try
 - catch
 - throw
 - throws
 - finally
- Whenever exception occur in java program then object of related exception class is created by JVM and it will be thrown to a catch block
- Then the catch block is called as an exception handler and it handles the exception object

Exception Handling By Using try ,catch

- We can handle the exceptions by using try catch.
- We have to place the risky code inside the try block and the corresponding exception handling code inside catch block.

```
try
{
    //Risky code
}
catch (Exceptionclassname e)
{
    //handling code
}
```

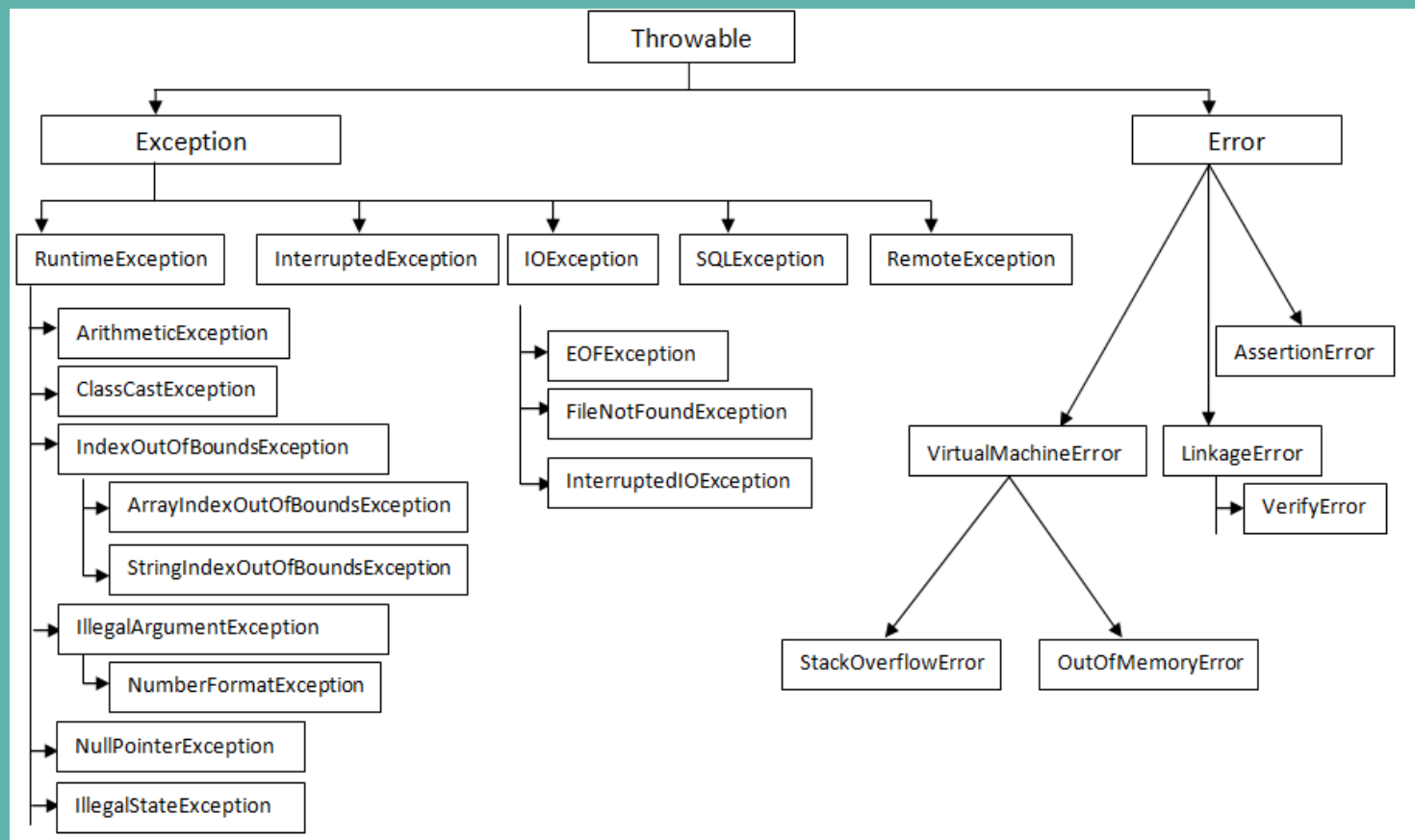
Exception Hierarchy

The Java exception hierarchy starts from Throwable class .
i.e for all java Exceptions and Errors Throwable is parent class.

Throwable has 2 sub classes

Exception: Most of the times, Exceptions are caused by due to our program code only and these are recoverable

Error: Most of the times errors are raised due to lack of system resources and Errors are non-recoverable.



Types of Exception :

1. Checked Exception
2. Unchecked Exception

Checked Exception:

- The Exception classes which are derived from `java.lang.Exception` class are called Checked Exception.
- They do not include `java.lang.RuntimeException` class and all its sub classes.
- All checked Exceptions must be handled by programmer explicitly by using try and catch blocks or throws clause otherwise compile-time error occurs.
- The java compiler checks try and catch blocks or throws clause for this kind of Exception.
- All application specific exceptions are comes under this category.

The Exceptions which are checked by the compiler for smooth execution of the program at run time are called **checked exceptions**.

Example: FileNotFoundException, InterruptedException.

2.Unchecked exceptions:

- The Exceptions which are unable to checked by the compiler are called '*unchecked exceptions*'
- ***RuntimeException*** and it's child classes, ***Error*** and it's child classes are considered as unchecked
- Exception and all the remaining classes considered as checked.
- The exception classes which are derived from `java.lang.RuntimeException` class are called Unchecked exceptions
- All unchecked exceptions are handled by JVM implicitly
- Handling unchecked exceptions by a programmer is optional
- The java compiler does not check for try catch or throws keyword for this kind of exceptions.
- All general exceptions are comes under this category

Example for Unchecked exception:

`public static int parseInt(String) throws NumberFormatException`

Ex: public class Demo

```
{  
    public static void main(String[] args)  
    {  
        int x=Integer.parseInt(args[0]);  
    }  
}
```

The above program successfully compiles

Finally

- It is not recommended to place cleanup code inside **try statement** because there is no guarantee for execution of all statements inside try block.
- It is not recommended to maintain cleanup code with in the **catch block** because if there is no exception the catch blocks won't be executed.
- We required a block to maintain cleanup code which should execute always irrespective of whether the exception is raised or not whether it is handled or not , such block is nothing but “*finally block*”
- Hence the main objective of finally block is to maintain cleanup code.

Multi Threading:

Execution of more than one thread at a time is called multithreading

Thread:

A thread is a piece of code that executes independently

There are two ways to create a thread

1. By extending `java.lang.Thread` class
2. By implementing `java.lang.Runnable` interface

Note: Every program contains atleast one thread i.e. main thread

Thread based Multitasking:

- Executing several tasks simultaneously where each task is a separate independent part of the same program is called Thread based Multitasking.
- Java provides in built support for thread based multitasking by providing rich library (`Thread`, `Runnable` ..etc)

Thread Scheduler:

- If multiple threads are there then which thread will get chance first for execution will be decided by “Thread Scheduler”.
- Thread Scheduler is the part of JVM.
- Note: whenever the situation comes to multithreading the guarantee in behavior is very less we can tell **possible output but not exact output**

Ex:

Mythread.java

The difference between `t.start()` & `t.run()`:

- In the case of `t.start()` a new thread will be created, which is responsible for the execution of `run()` method.
- But in the case of `t.run()` no new thread will be created and `run()` method will be executed just like a normal method by the main thread.

Importance of Thread Class `start()` method:

- After Creating thread object compulsory we should perform registration with the thread scheduler. **This will take care by `start()` of Thread class**
- With out executing Thread class `start()` method there is no chance of start a new Thread in java.

If we are not overriding run() method:

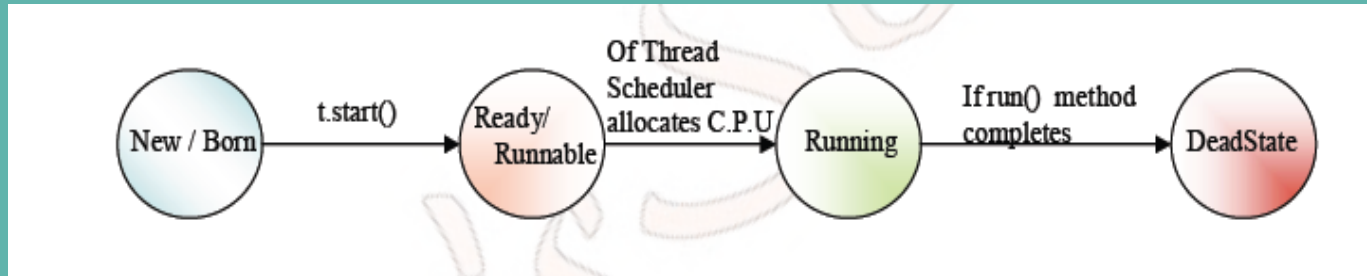
Then the thread class run method will be executed which has empty implementation.

Ex:

```
class MyThread extends Thread
{
}
class ThreadDemo
{
    public static void main(String arg[])
    {
        MyThread t = new MyThread();
        t.start();
    }
}
```

O/P:- no output

Life cycle of Thread



```
new MyThread();
```

- Once we created a thread object then it is said to be in **new state** or born state
- If we call `start( )` method then the thread will be entered into **ready or runnable state**
- If thread scheduler allocates cpu then the thread will be entered into **running state**
- If `run( )` method completes its execution then the thread will be entered into **deadstate**

Thread class

fields:

```
public static final int MIN_PRIORITY;  
public static final int NORM_PRIORITY;  
public static final int MAX_PRIORITY;
```

Constructors:

```
public java.lang.Thread();  
public java.lang.Thread(java.lang.Runnable);
```

Methods:

1. `public static native java.lang.Thread currentThread();`

it is used to get the current thread object

2. `public static native void sleep(long) throws java.lang.InterruptedException;`
sleep causes the current thread to suspend execution for a specified period.

3. `public synchronized void start();`

It is used to call `run()` method

4. `public void run();`

It is a null body method

5. `public void destroy();`

The **destroy() method** of thread class is **used to destroy** the thread group and all of its subgroups.

6. `public final void setPriority(int);`

The `setPriority()` method of thread class is used **to change the thread's priority**

7. `public final int getPriority();`

The `getPriority()` method of thread class is used **to check the priority of the thread**

8. `public final synchronized void setName(java.lang.String);`

The **`setName()` method** of thread class is **used** to change the name of the thread

9. `public final java.lang.String getName();`

method is used **to get the name of a thread.**

Java.lang.Runnable:

It is an interface which is implemented by any class.

We can define a Thread even by implementing Runnable interface also. Runnable interface present in java.lang. package and contains only one method run().

```
Public abstract void run();
```

Defining a thread By Implementing Runnable Interface

- We can define a Thread by implementing runnable interface also.
- Runnable interface available in java.lang package and contains only one method.

```
public void run();
```

Note: Among 2- approaches of defining a thread – implements Runnable is always recommended to use

Thread Priorities:

Every Thread in java having some priority.

The range of valid thread priorities is (1-10) (1 is least & 10 is Highest).

Thread class defines the following constant for representing some standard priorities.

Thread.MIN_PRIORITY → 1

Thread.NORM_PRIORITY → 5

Thread.MAX_PRIORITY → 10

- Thread scheduler use these priorities while allocating C.P.U. The Thread which is having highest priority will get chance first for execution.
- If two threads having the same priority then which thread will get chance first for execution is decided by Thread Scheduler, which is vendor (Operating System)dependent
- The default priority for the main thread is 5
- For all the remaining threads the priority will be inherited from parent.

Thread class contains the following methods to set and get priorities of thread.

```
public final void setPriority(int priority)
```

```
public final int getPriority();
```

where priority should be from 1-10 other wise R.E: `IllegalArgumentException`.

In the above example If we are commenting line 1 then both main and child threads having the same priority(5) and hence we cant expect exact execution order and exact output

If we are not commenting the method then main has the priority 5 and child thread have the priority(10) and hence child thread will be executed first and then main thread.

Note: Some Operating Systems may not provide support for thread priorities.

The methods to prevent thread execution

To prevent a thread from execution following methods are used.

- 1) yield()
- 2) join()
- 3) sleep()

yield()

The thread which is called yield() method temporarily *pause* the execution to give the chance for remaining threads of same priority.

```
public static native void yield();
```

The thread which is yielded, when it will get chance once again for execution is decided by ThreadScheduler and we can't expect exactly.

- If we are commenting line 1 then both threads will be executed simultaneously and we cant expect exact execution order
- If we are not commenting line 1 then the chance of completing main thread first is high because child thread always calls yield() method.
- **As the yield method is native method (implemented in some other programming language like c, c++) some Operating system may not provide the support for this.**

join()

If a thread wants to wait until some other thread completion then we should go for join method.

Ex:

If a thread t1 executes t2.join(), then t1 will be entered into waiting state until t2 completion.

```
public final void join() throws InterruptedException  
public final void join(long ms) throws InterruptedException  
public final void join(long ms, int ns) throws InterruptedException
```



- **Join() method is overloaded and every join() throws InterruptedException.**
- **Hence when ever we are using join() compulsory we should handle InterruptedException either by using try-catch block or by throws key word otherwise we will get compile time error.**

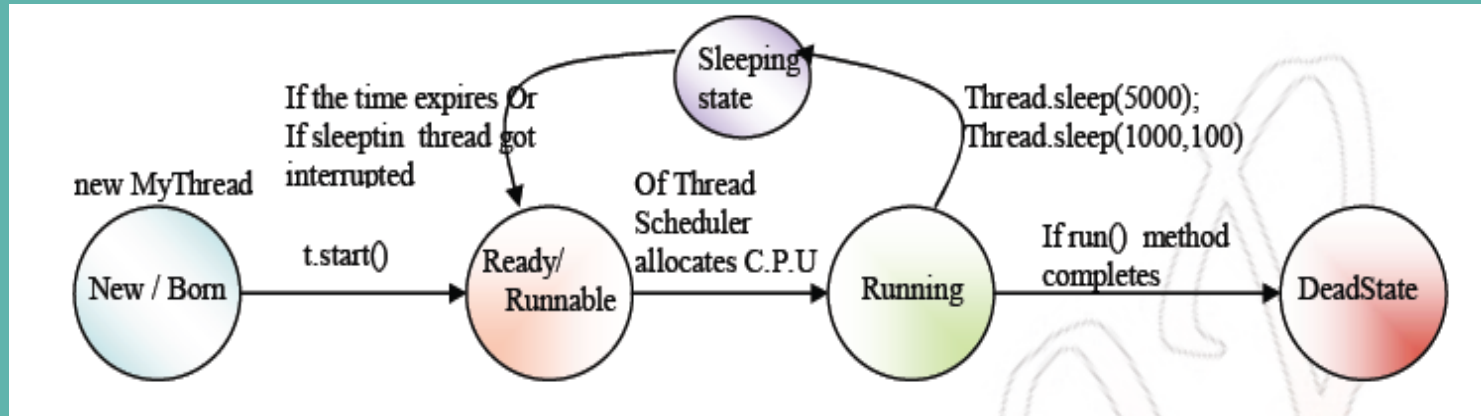
- If we are commenting line 1 then both threads will be executed simultaneously and we cant expect exact execution order and hence we cant expect exact o/p.
- If we are not commenting line 1 then main thread will wait until completing child thread hence in this case the o/p is expected.

Sleep ():

- If a thread has to wait for some predefined amount of time with out execution then we should go for sleep() method.
- If a thread don't want to perform any operation for a particular amount of time then we should go for sleep()

- `public static void sleep(long ms) throws InterruptedException`
- `public static void sleep(long ms, int ns) throws InterruptedException`

When ever we are using `sleep()` compulsory we should handle **InterruptedException** other wise we will get compile time error



Synchronization

- ❑ 'synchronized' is the keyword applicable for the *methods* and *blocks*. We can't apply this keyword for *variables* and *classes*.
- ❑ If a method declared as synchronized at a time only one thread is allowed to execute that method on the given object.
- ❑ The main advantage of synchronized keyword is we can overcome data inconsistency problem.
- ❑ The main limitation of synchronized keyword is it increases the waiting time of a thread, and effects the performance of the system.
- ❑ Every object in java has a unique lock.

Synchronization

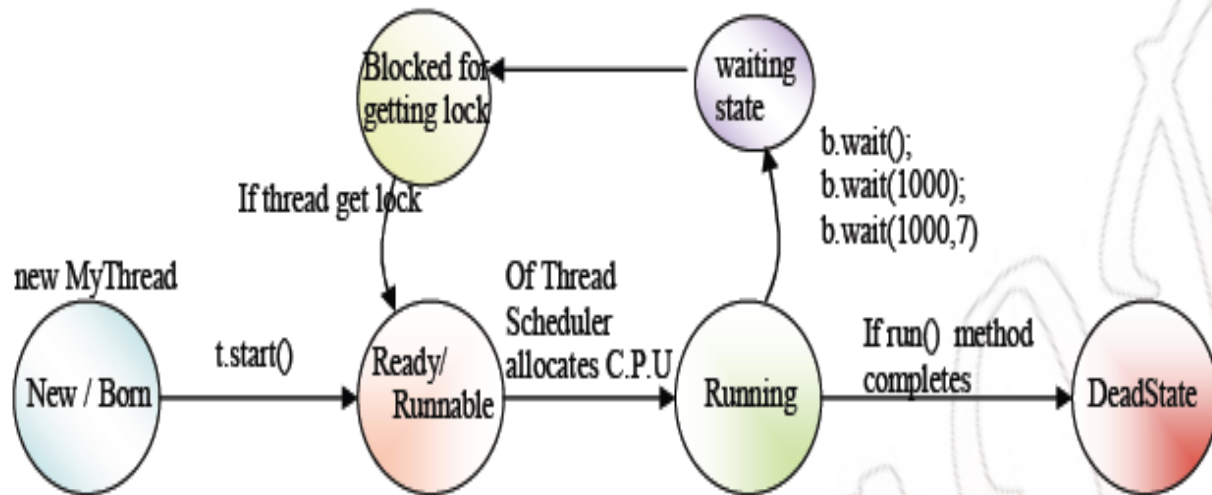
- ❑ Every object in java has a unique lock.
- ❑ Synchronization concept internally implemented by using this lock concept.
- ❑ If a thread want to execute any synchronized method on the object first it should require the lock of that object.
- ❑ Once a thread got the lock then it is allowed to execute any synchronized method on that object.
- ❑ Once synchronized method completes then automatically the lock will be released
- ❑ While a thread executing a synchronized method on the object, then the remaining threads are not allowed to execute any synchronized method on the same object.

Synchronized Blocks

- It is not recommended to declare entire method as synchronized.
- If very few lines of code requires the synchronization concept.
- Then Such a few lines of code we can declare in a block rather than declaring entire method as a synchronized method

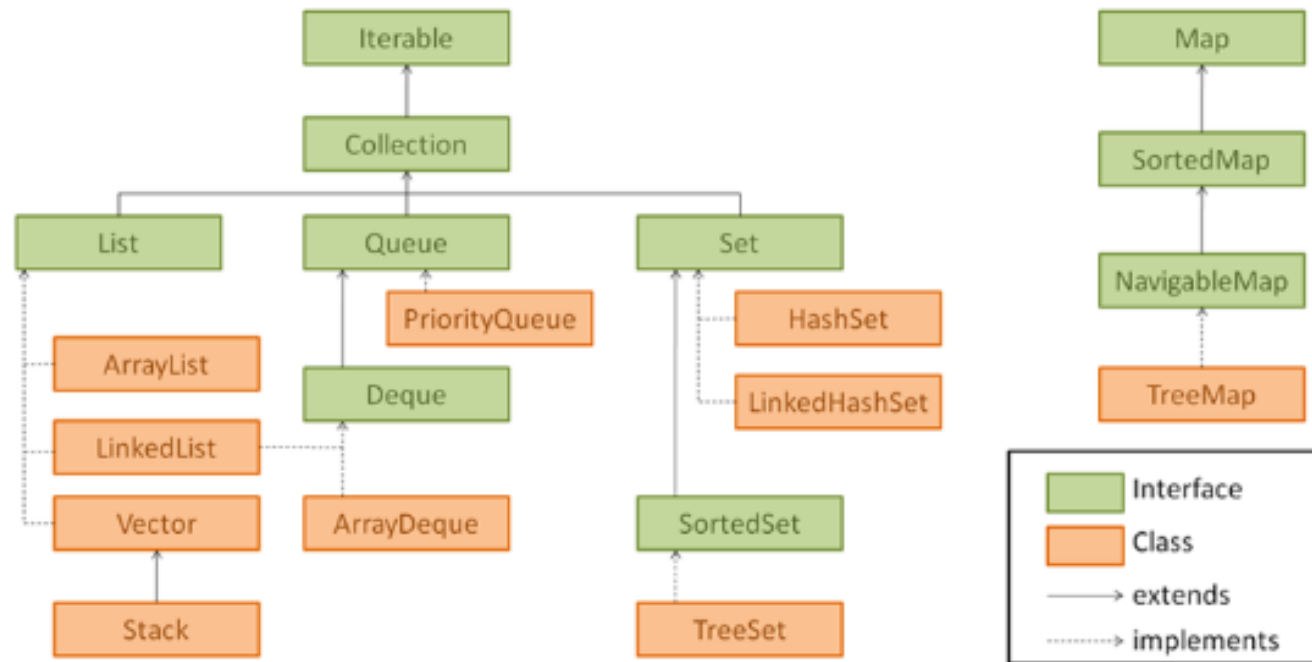
Inter Thread Communication

- ❑ Two threads can communicate with each other by using wait(), notify(), notifyAll().
- ❑ These methods are available in Object class but not in thread class.
- ❑ If a thread executes wait() method it immediately releases the lock of that object and entered into waiting state.
- ❑ After giving the notification also the thread releases the lock but may not be immediately.



Collections Framework:

- A collection is an object that represents a group of objects
- The Java **collections** framework provides a set of interfaces and classes to implement various data structures and algorithms.
- **Ex:** LinkedList class of the collections framework provides the implementation of the doubly-linked list data structure.
- The Java collections framework provides various interfaces.
- These interfaces include several methods to perform different operations on collections.
- Java Collections can achieve all the operations that you perform on a data such as searching, sorting, insertion, manipulation, and deletion.
- Java Collection framework provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).



Collection (interface):

- The Collection interface is **the root interface of the Java collections framework**.
- The Collection interface is not directly implemented by any class.
- This interface has 3 sub-interfaces List, Set, and Queue.
- It defines the methods that are commonly used by almost all the collections.
- The Collection interface extends **Iterable** interface.

java.util.Collection

- **boolean add(Object obj)**
- Adds the **obj** to the invoking collection.
- **boolean addAll(Collection c)**
- Adds all the elements of **c** to the invoking collection.
- **boolean remove(Object obj)**
- Removes the **obj** from the invoking collection.
- **boolean removeAll(Collection c)**
- Removes all the elements of **c** from the invoking collection.
- **boolean retainAll(Collection c)**
- Removes all the elements from the invoking collection except elements in **c**.
- **void clear()**
- Removes all elements from the invoking collection.
- **boolean contains(Object obj)**
- Finds the **obj** to the invoking collection.

- **boolean containsAll(Collection c)**
 - Finds all elements of **c** in the invoking collection.
- **boolean equals(Object obj)**
 - Returns true if the **obj** and the invoking collection are equal, else returns false.
- **boolean isEmpty()**
 - Returns true if the invoking collection is empty, else returns false.
- **int size()**
 - Returns total number of elements in the invoking collection.
- **int hashCode()**
 - Returns hash code for the invoking collection.
- **Object[] toArray()**
 - Returns an array that contains all elements of the invoking collection.
- **Object[] toArray(Object obj)**
 - Returns an array that contains only those elements whose type matches the array type.

List interface

- The **List** interface is a child interface of the Collection interface.
- The List interface is available inside the **java.util** package.
- List implemented classes, can store the collection of elements and each element stored on the basis of Index.
- It can contain duplicate elements as well.
- Java List interface allows us to have 'null' elements in the Java list
- The **List interface** inherits the methods from Collection interface. The **List interface** has its implementation in various classes like ArrayList, LinkedList, Vector and Stack.

1. **int size()**
2. **boolean isEmpty()**
3. **boolean contains(Object o)**
4. **Iterator iterator():** To get an iterator so that you can iterate over the elements in the list in the proper sequence.
5. **Object[] toArray():** It returns an array that contains all the elements of the list in proper sequence
6. **get(int index):** To get the element from the specified position in the list.
7. **set(int index, E element):** This method is used to add the element at a specific position. If any element is already there, then it Replaces the element at the specified position.
8. **ListIterator listIterator():** It returns a list iterator over the elements in the list.
9. **List subList(int fromIndex, int toIndex)**

ArrayList

- ✓ The underlying data Structure for ArrayList() is resizable Array or “*Growable Array*”.
- ✓ Duplicate objects are allowed.
- ✓ Insertion order is preserved.(insertion happens based on index value)
- ✓ Heterogeneous objects are allowed.
- ✓ ‘null’ insertion is possible.
- ✓ Without **generics**, ArrayList allows to insert **any type of objects**;

Constructors of ArrayList

```
ArrayList AL = new ArrayList()
```

Creates an empty ArrayList object with default initial capacity 10.
When ever ArrayList reaches its max capacity a new ArrayList Object will be created with new capacity.

```
ArrayList l = new ArrayList(int initial capacity)
```

Creates an empty ArrayList Object with the specified initial capacity.

```
ArrayList l = new ArrayList(Collection c)
```

For inter conversion between collection objects.

- ArrayList and vector classes implement RandomAccess interface,so that we can access any element with the same speed.
- ArrayList is best suitable if our frequent operation is retrieval operation.
- ArrayList is the worst choice if user want to perform insertion or deletion in the middle.

LinkedList

- The underlying Data Structure for linked list is doubly linked list.
- Duplicate objects are allowed.
- Insertion order is preserved.
- Heterogeneous objects are allowed.
- 'null' insertion is possible.
- Implements List, Queue, serializable, cloneable Interfaces But not RandomAccess.
- LinkedList is the best choice if our frequent operation is insertion or deletion in the middle

- LinkedList is the worst choice if our frequent operation is retrieval operation.
- LinkedList class usually used for implementing stacks and Queues
- LinkedList class specific methods are:

1. void addFirst(Object obj)
2. void addLast(Object obj)
3. Object removeFirst()
4. Object removeLast()
5. Object getFirst()
6. Object getLast()

Constructors

```
LinkedList l = new LinkedList()
```

```
LinkedList l = new LinkedList(Collection c)
```

Inter conversion between collection objects.

```
ArrayList l1 = new ArrayList();
```

```
l1.add(10);
```

```
l1.add(20);
```

```
System.out.println("l1--->" + l1);
```

```
LinkedList l2 = new LinkedList(l1);
```

```
l2.add(1,5);
```

```
l2.add(3,5);
```

```
System.out.println("l2--->" + l2);
```

```
ArrayList l3 = new ArrayList(l2)
```

```
System.out.println("l3--->" + l3);
```

VectorClass

The underlying Data structure for the vector is resizable array or growable array.

Insertion order is preserved.

Duplicate objects are allowed.

'null' insertion is possible.

Heterogeneous objects are allowed.

Best choice if the frequent operation is retrieval.

Worst choice if the frequent operation is insertion or deletion in the middle.

Vector class implemented serializable, cloneable and RandomAccess Interfaces.

Stack

- It is the child class of Vector contains only one constructor.

```
Stack s = new Stack();
```

Methods:

- Object push(Object obj);
For inserting an object to the stack
- Object pop();
It removes and returns top of the stack.
- Object peek();
Returns the top of the stack without removal of object.
- int search(Object obj);
If the specified object is available it returns its offset from top of the stack and If the object is not available then it returns -1.
- boolean empty();
returns true if the stack is empty otherwise false.

Cursors in collection framework

- To visit objects one by one from Collection we have to use cursors.
- Cursor is also called as Iterator.
- From the collection object to retrieve objects we can use the following 3 cursors.
 1. Enumeration
 2. Iterator
 3. ListIterator

Enumeration

This interface has introduced in 1.0 version it contains the following 2 methods.

```
boolean hasMoreElements();  
Object nextElement();
```

Limitations of Enumeration

- It is applicable only for legacy classes and it is not a universal cursor.
- While iterating the elements by using enumeration we can perform only read operation and we can't perform any modify/removal operations.
- To overcome these problems we should go for Iterator interface.

Iterator

- Introduced in 1.2 version.
- We can use Iterator Object for any collection implemented class i.e it is universal cursor.
- By iterating the elements we can perform remove operation also in addition to read operation.

This interface contains the following 3 methods.

1. boolean hasNext();
2. Object next();
3. void remove();

Note:

- ❑ Enumeration and Iterator are single directional cursors. They can always move towards forward direction only.
- ❑ By using Iterator we can perform read and remove operations.
- ❑ And we can't perform any replace or addition of new objects
- ❑ To overcome these limitations we should go for ListIterator interface.

ListIterator

- 1.It has introduced in 1.2 version and it is child interface of Iterator.
- 2.It is a bidirectional cursor.
- 3.i.e Based on requirement we can move either to the forward or backward direction.
- 4.While Iterating we can perform replace and add operation in addition to read and remove this

ListIterator interface defines the following 9 methods.

1. boolean hasNext();
2. boolean hasPrevious();
3. Object next();
4. Object previous();
5. void remove();
6. void set(Object)

Note:

nextIndex(): If there is no next element it returns size of the list.

previousIndex(): If there is no previous element it returns -1

The most powerful cursor is ListIterator.

But it's main limitation is it is applicable only for list implemented classes (ArrayList, LinkedList, Vector, Stack).

Set Interface:

- This can be used for representing a group of Individual objects where Insertion order is not preserved
- Duplicate objects are **not allowed**.
- Set interface is child interface of Collection.
- This interface doesn't contain any new method it contains all the Collection interface methods

HashSet Class:

- ✓ The underlying Data Structure for HashSet is Hashtable.
- ✓ Insertion order is not preserved and it is based on hash code of the Object.
- ✓ Duplicate objects are not allowed.

HashSet Methods:

boolean add(Object o)

- Adds the specified element to this set if it is not already present.
- This method simply returns false. If the object is already existing
- 'null' insertion is possible.
- Heterogeneous objects are allowed.
- HashSet is the best choice if the frequent operation is Search Operation.

Constructors:

1. HashSet h = new HashSet()
2. HashSet h = new HashSet(int initialcapacity)
3. HashSet h = new HashSet(Collection c)

SortedSet

Represents a group of individual objects where duplicate objects are not allowed. Insertion order is not preserved but all the elements are inserted according to some sorting order of elements.

The sorting order may be default natural sorting order or customized sorting order. SortedSet Interface contains the following more specific methods

1. Object first()

Returns first element in the SortedSet

2. Object last()

Returns last element in the SortedSet

3. SortedSet headSet(Object obj)

Returns the SortedSet contains the elements which are less than the specified object obj.

4. SortedSet tailSet(Object obj)

Returns the SortedSet whose elements are greater than or equal to the specified object obj

5. SortedSet subSet(Object obj1, Object obj2)

Returns the SortedSet whose elements are $\geq$ obj1 but $<$ obj2

6. Comparator comparator();

TreeSet :

- The underlying Data structure for the TreeSet is Balanced tree.
- Duplicate objects are not allowed.
- If we are trying to insert duplicate object add method simply returns false.
- Insertion order is not preserved but all the elements are inserted according to some sorting order.
- Heterogeneous objects are not allowed, violation leads to Run time error saying class cast Exception

Constructors :

1. `TreeSet t = new TreeSet();`

Creates an empty TreeSet Object where the sorting order is default natural sorting order.

2. `TreeSet t= new TreeSet(Comparator c)`

Creates an empty TreeSet Object where the Sorting order is specified by comparator object.

3. `TreeSet t= new TreeSet(Collection c)`

4. `TreeSet t= new TreeSet(SortedSet s)`

Creates TressSet Object for a given SortedSet.

Map:

- 1) Map is not child interface of Collection.
- 2) If we want to represent a group of objects as key-value pairs then Map is useful
- 3) Duplicate keys are not allowed but values can be duplicated.
- 4) Each key-value pair is called entry
- 5) There is no relation between Collection and Map
- 6) Collection meant for group of individual objects and Map is meant for group of key-value pairs

Each key-value pair considered as Entry object, hence Map is nothing but a set of entries.

Methods of Map interface:

1.Object put(Object key,Object value);

- To add key-value pair to the map
- If the specified key is already available then old value will be replaced with new value and old value will be returned.
- If the entry is added to the map then this method will return null

2.void PutAll(Map m)

To add a group of key-value pairs

3.Object get(Object key)

Returns the value associated with specified key

If the key is not available then we will get null

Methods of Map interface:

4.Object remove(Object key)

5.boolean containsKey (Object key)

6.boolean containsValue (Object key)

7.int size();

8.boolean isEmpty();

9.Void clear()

10. Set keySet();// The set of keys can be obtained.

11. Collection values(); //The set of values can be obtained.

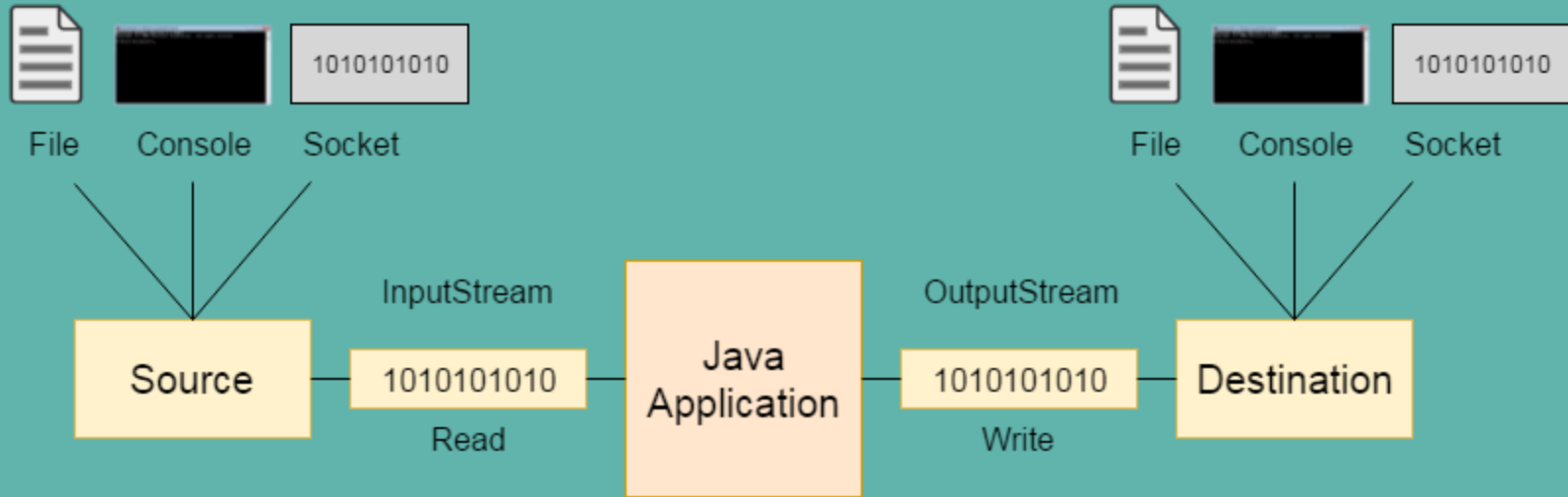
12. Set entrySet(); // The set of entryset can be obtained.

The java IO package(Java Streams):

- Java uses the concept of stream to make I/O operation .
- The java.io package contains all the classes required for input and output operations.
- A stream is a flow of data from source to destination
- A Source can be a keyboard, file, server, client...etc
- A destination can be monitor, file, client , server...etc
- stream-based IO operations are performed using two separate streams input stream and output stream
- The java stream is composed of bytes.
- In java, streams are divided into three categories,
- 1.Console Input/Output streams
- 2.File Input/Output streams
- 3.Network Input/output Streams

InputStream:

Java application uses an input stream to read data from a source(keyboard, file, socket)



OutputStream:

`OutputStream` class is an abstract class. It is the superclass of all classes representing an output stream of bytes.

Types of Streams

Depending upon the data a stream holds, it can be classified into:

- ✓ Byte Stream
 - ✓ Character Stream
-
- All byte stream classes are derived from base abstract classes called **InputStream** and **OutputStream**.
 - Byte Stream classes handle text,image,graphics,animation, audio, video data.
-
- All the character stream classes are derived from base abstract classes **Reader** and **Writer**.
 - Character stream classes handle only text

Serialization:

- It is a process of converting Object into series of bits.
- In java Object must be serializable to do the following operations

1. Writing object to a file
2. Writing object to a network
3. Reading object from a file
4. Reading object from a network

- Class must be marked with `java.io.Serializable` interface to make serializable objects
- `Java.io.Serializable` interface is called as marker interface or tag interface or empty interface because it does not contain members.

Inner classes :

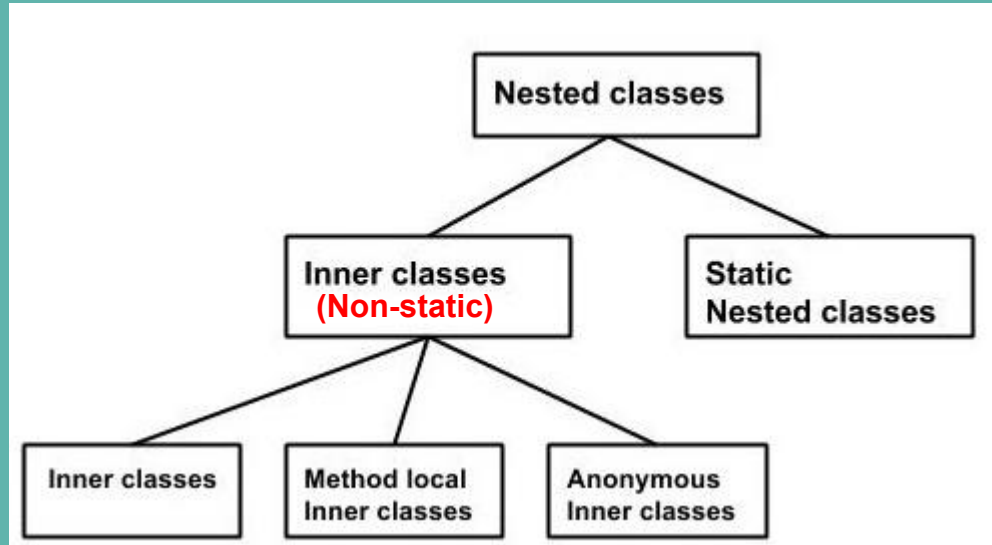
In Java, we can define a class within another class. Such class is known as nested class

(or)

Java inner class or **nested class** is a class that is declared inside the class or interface.

Syntax:

```
class OuterClass
{
// ...
class NestedClass
{
// ...
}
}
```



1. Inner class:

A class that is defined as a member of another class is known as Inner class or Member class.

Ex:

```
Class A
```

```
{
```

```
    Class B{
```

```
        void show(){
```

```
            System.out.println("Member class");
```

```
        }
```

```
    }
```

```
Class Demo{
```

```
    Public static void main(String args[]){
```

```
        A a = new A();
```

```
        a.new B().show();
```

```
    }
```

```
}
```


2. Static Inner class:

A class that is defined as a static member of another class is known as “Nested Top Level class”.

Ex:

Class A

```
{  
    Static Class B{  
        void show(){  
            System.out.println("Static class");  
        }  
    }  
}  
Class Demo{  
    Public static void main(String args[]){  
        A.B.show();  
    }  
}
```

3. Method Local Inner class:

A class that is defined inside method is known as Local class.

Ex:

Class A

{

Void show(){

Class B{

void print(){

System.out.println("Local class");

}}

B ob = new B();

Ob.print();

}}

Class Demo{

Public static void main(String args[]){

A ob = new A(); ob.show();

}

}

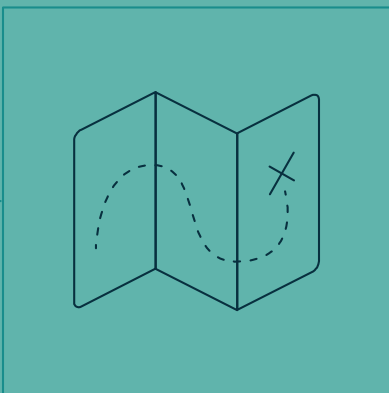
4. Anonymous Inner class:

It is one type inner class which has no name.

It is always sub class of a Interface.

Ex:

```
Interface A{  
Void show();  
}  
Class Demo{  
Public static void main(String args[]){  
A ob = new A(){  
Public void show(){  
System.out.println("Anonymous class");  
}  
};  
Ob.show();  
}  
}
```



Thanks!

Any questions?

You can find me at sayeedunnisa.js@gmail.com